

LAB FILE
QUANTUM COMPUTING
(CSBB 304)

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE AND ENGINEERING

Submitted By
AASHISH KHANIYA
(241210002)

Submitted To
DR. V S PANDEY (ASSISTANT PROFESSOR, DoCSE)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
NATIONAL INSTITUTE OF TECHNOLOGY DELHI
MAY 2026

Lab Report: Introduction to Qiskit and Single-Qubit Gates

1. Objective

The goal of this lab is to:

- Introduce the Qiskit framework for quantum computing.
 - Create and simulate simple quantum circuits.
 - Understand and apply single-qubit quantum gates: **Pauli-X**, **Pauli-Y**, **Pauli-Z**, and **Hadamard (H)**.
 - Visualise quantum states using the **Bloch Sphere**, **statevectors**, and **unitary matrices**.
 - Observe quantum measurements using the **qasm_simulator**.
-

2. Components

- **Software:** Jupyter Notebook, Python 3.10, Qiskit 2.1.1, Qiskit Aer 0.17.1, Matplotlib 3.10.5
 - **Qiskit Simulators Used:** statevector_simulator, unitary_simulator, qasm_simulator
 - **Qiskit Circuit Elements:** QuantumCircuit, QuantumRegister, ClassicalRegister
 - **Quantum Gates Used:** X-gate (Pauli-X), Y-gate (Pauli-Y), Z-gate (Pauli-Z), H-gate (Hadamard)
-

3. Architecture

A quantum circuit in Qiskit comprises:

- **Quantum Register (qr):** A collection of qubits initialised to $|0\rangle$.
- **Classical Register (cr):** Stores results of qubit measurements.
- **QuantumCircuit:** Applies a sequence of operations (gates, measurements) to the qubits.

Each gate modifies the quantum state, and its effect is observed using:

- **Statevector:** Full description of the quantum state.
 - **Unitary matrix:** Mathematical representation of the transformation.
 - **Measurement:** Collapses the quantum state into a classical bitstring.
-

4. Procedure

Step 1: Environment Setup

Installed Qiskit, Qiskit Aer, and Matplotlib using pip.

```
%pip install qiskit qiskit-aer matplotlib
```

Step 2: Initialisation

Imported necessary modules, created quantum and classical registers, and initialised circuits.

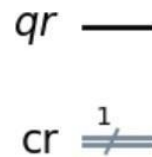
```
import qiskit
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator, Aer
from qiskit.visualization import plot_histogram
import matplotlib
from math import pi, sqrt # pi = 3.14 and square root operation
from qiskit.visualization import plot_bloch_multivector # Plotting the Bloch Sphere for Single Qubits
from qiskit.visualization import plot_histogram # 2D Histogram Plotting
```

```
from qiskit import ClassicalRegister, QuantumRegister, QuantumCircuit
```

Basic quantum circuit:

```
cr = ClassicalRegister(1,"cr")
qr = QuantumRegister(1,"qr")
qc = QuantumCircuit(qr,cr)
```

```
qc.draw(output='mpl')
```



Step 3: Pauli-X Gate

Theory:

- Flips qubit: $|0\rangle \rightarrow |1\rangle$ and vice-versa.
- Matrix:

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

Simulations:

- **Statevector:** $[0.+0.j, 1.+0.j] \rightarrow |1\rangle$

```
# Pauli-X gate on |0> qubit
qr_x = QuantumRegister(1, 'qr_x')
cr_x = ClassicalRegister(1, 'cr_x')

qc_x = QuantumCircuit(qr_x)

# apply X-gate to the 0th qubit
qc_x.x(qr_x[0])
# qc.save_statevector()
# qc.save_unitary()
qc_x.draw(output='mpl')
```

- **Bloch Sphere:**

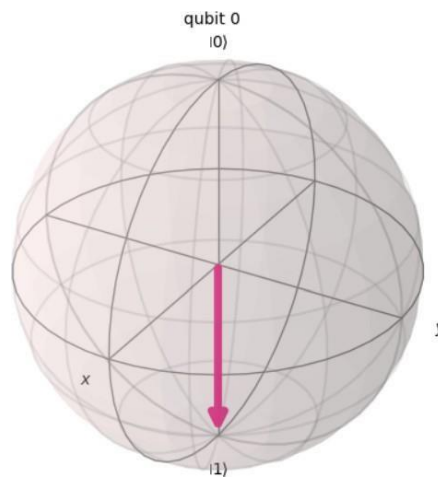
```
backend = Aer.get_backend('statevector_simulator')
transpiled_qc_x = transpile(qc_x, backend)
result = backend.run(transpiled_qc_x).result().get_statevector()

print("Result after implementing x-gate:", result)

plot_bloch_multivector(result)
```



Result after implementing x-gate: Statevector([0.+0.j, 1.+0.j],
dims=(2,))



- **Unitary:** Matches expected matrix.

```
backend = Aer.get_backend('unitary_simulator')
result = backend.run(transpiled_qc_x).result().get_unitary(qc_x, decimals=3)
print("Unitary matrix after implementing x-gate:\n", result)
```

Unitary matrix after implementing x-gate:
Operator([[0.+0.j, 1.+0.j],
[1.+0.j, 0.+0.j]],
input_dims=(2,), output_dims=(2,))

- **Measurement:** 1024/1024 shots yielded '1'.

```
# measuring the qubit |0> after applying X-gate

cr_x = ClassicalRegister(1, 'classical register')
qr_x = QuantumRegister(1, 'quantum register')
qc_x = QuantumCircuit(qr_x, cr_x)

qc_x.x(qr_x[0])

qc_x.measure(qr_x, cr_x) # measure the qubit

qc_x.draw(output='mpl')
```



- **Pauli X-Gate on QASM Simulator:**

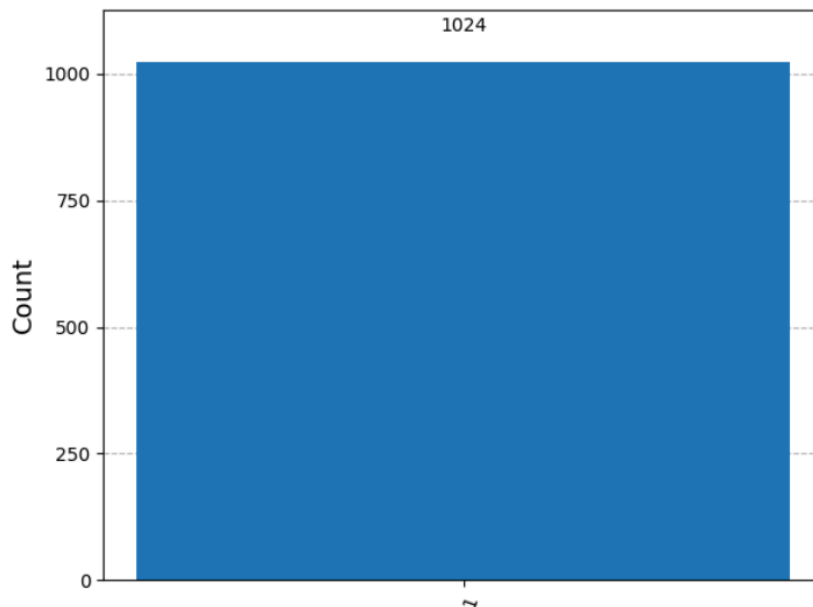
```
backend = Aer.get_backend('qasm_simulator')
job = transpile(qc_x, backend)

result = backend.run(job).result()
counts = result.get_counts(qc_x)

print("Counts after applying X-gate and measuring the qubit:", counts)

plot_histogram(counts)
```

Counts after applying X-gate and measuring the qubit: {'1': 1024}



Step 4: Pauli-Y Gate

Theory:

- Adds imaginary phase: $|0\rangle \rightarrow i|1\rangle$, $|1\rangle \rightarrow -i|0\rangle$
- Matrix:

$$Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$$

Simulations:

- **Statevector:** $[0.+0.j, 0.+1.j] \rightarrow i|1\rangle$

```
# Pauli-Y gate on |0> qubit
qr_y = QuantumRegister(1, 'qr_y')
cr_y = ClassicalRegister(1, 'cr_y')

qc_y = QuantumCircuit(qr_y)

# apply Y-gate to the 0th qubit
qc_y.y(qr_y[0])
# qc.save_statevector()
# qc.save_unitary()
qc_y.draw(output='mpl')
```



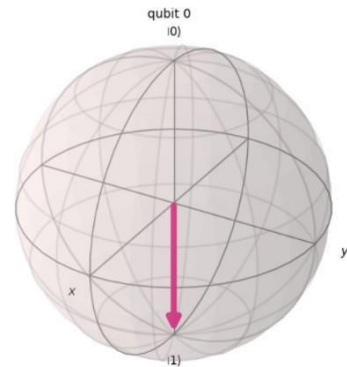
- **Bloch Sphere:**

```
backend = Aer.get_backend('statevector_simulator')
transpiled_qc_y = transpile(qc_y, backend)
result = backend.run(transpiled_qc_y).result().get_statevector(qc_y, decimals=3)

print("Result after implementing y-gate:\n",result)

plot_bloch_multivector(result)
```

Result after implementing y-gate:
 Statevector([0.-0.j, 0.+1.j],
 dims=(2,))



- **Unitary:** Matches theoretical Y matrix.

```
backend = Aer.get_backend('unitary_simulator')
result = backend.run(transpiled_qc_y).result().get_unitary(qc_y, decimals=3)
print("Unitary matrix after implementing y-gate:\n",result)
```

Unitary matrix after implementing y-gate:
 Operator([[0.-0.j, 0.-1.j],
 [0.+1.j, 0.+0.j]],
 input_dims=(2,), output_dims=(2,))

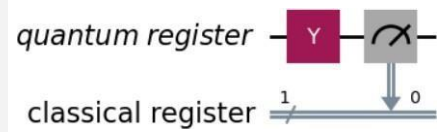
- **Measurement:** 1024/1024 shots yielded '1'.

```
# Measuring the qubit |0> after applying Y-gate
qr_y = QuantumRegister(1, 'quantum register')
cr_y = ClassicalRegister(1, 'classical register')
qc_y = QuantumCircuit(qr_y, cr_y)

qc_y.y(qr_y[0])

qc_y.measure(qr_y, cr_y)    # measure the qubit

qc_y.draw(output='mpl')
```



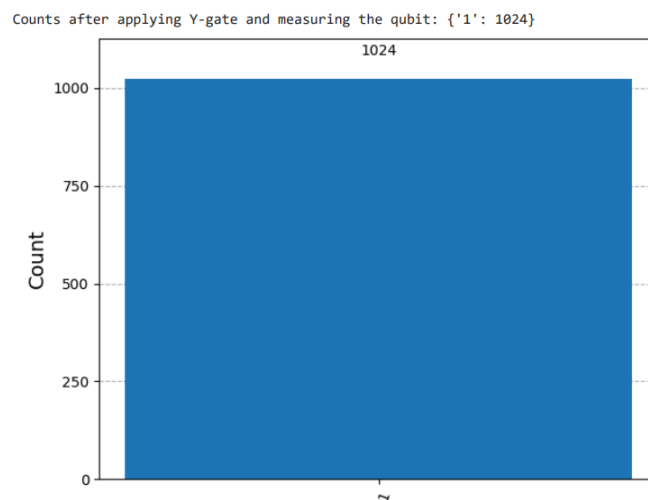
- **QASM Simulator:**

```
backend = Aer.get_backend('qasm_simulator')
job = transpile(qc_y, backend)

result = backend.run(job).result()
counts = result.get_counts(qc_y)

print("Counts after applying Y-gate and measuring the qubit:", counts)

plot_histogram(counts)
```



Step 5: Pauli-Z Gate

Theory:

- Applies phase flip: $|0\rangle \rightarrow |0\rangle$, $|1\rangle \rightarrow -|1\rangle$
- Matrix:

$$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

Simulations:

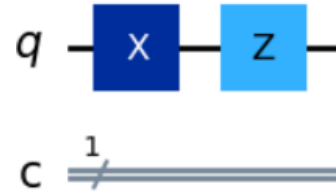
- Applied **X**, then **Z** to flip and apply phase.

- **Statevector:** $[0.+0.j, -1.+0.j] \rightarrow -|1\rangle$

$$|0\rangle \xrightarrow{\bar{X}} |1\rangle \xrightarrow{Z} -|1\rangle.$$

```
# Pauli-Y gate on |0> qubit
qc_z = QuantumCircuit(1,1,name='qc_z')

# apply x-gate to the 0th qubit
qc_z.x(0)
# apply z-gate to the 0th qubit
qc_z.z(0)
# qc.save_statevector()
# qc.save_unitary()
qc_z.draw(output='mpl')
```



- **Bloch Sphere:**

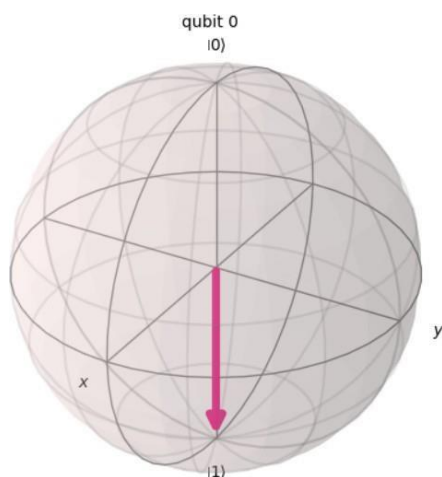
```
backend = Aer.get_backend('statevector_simulator')
transpiled_qc_z = transpile(qc_z, backend)
result = backend.run(transpiled_qc_z).result().get_statevector(qc_z, decimals=3)

print("Result after implementing z-gate:\n",result)

plot_bloch_multivector(result)
```

Caption

Result after implementing z-gate:
Statevector([0.+0.j, -1.-0.j],
dims=(2,))



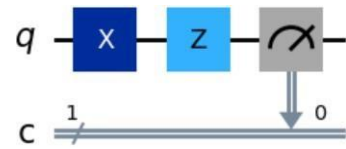
- **Measurement:** 1024/1024 shots yielded '1'.


```
# Measuring the qubit  $|0\rangle$  after applying Y-gate
```

```
qc_z = QuantumCircuit(1,1,name='qc_y')
qc_z.x(0)
qc_z.z(0)

qc_z.measure(0,0) # measure the qubit

qc_z.draw(output='mpl')
```



- **Unitary Gate:**

```
backend = Aer.get_backend('unitary_simulator')
result = backend.run(transpiled_qc_z).result().get_unitary(qc_z, decimals=3)
print("Unitary matrix after implementing z-gate:\n",result)
```

Unitary matrix after implementing z-gate:

```
Operator([[ 0.+0.j,  1.+0.j],
          [-1.-0.j,  0.+0.j]],
         input_dims=(2,), output_dims=(2,))
```

- **QASM Simulator:**

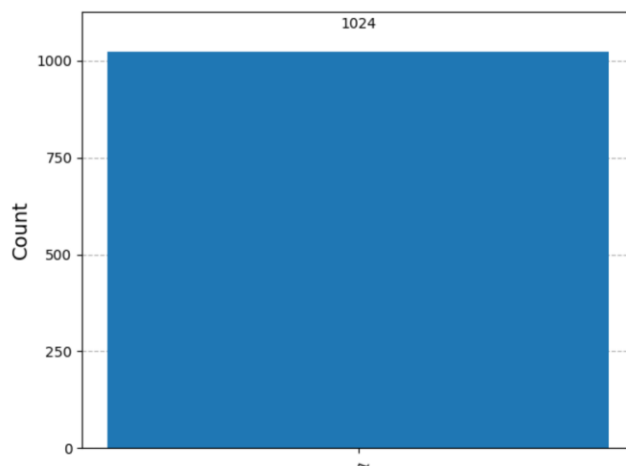
```
backend = Aer.get_backend('qasm_simulator')
job = transpile(qc_z, backend)

result = backend.run(job).result()
counts = result.get_counts(qc_z)

print("Counts after applying Y-gate and measuring the qubit:",counts)

plot_histogram(counts)
```

Counts after applying Y-gate and measuring the qubit: {'1': 1024}



Step 6: Hadamard Gate (H)

Theory:

- Creates superposition:

$$|0\rangle \rightarrow \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = |+\rangle$$

$$|1\rangle \rightarrow \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = |-\rangle$$

- Matrix:

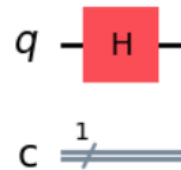
$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

Simulations:

- Statevector:** [0.707+0.j, 0.707+0.j] → Equal superposition

```
# H-gate on |0> qubit
qc_h = QuantumCircuit(1,1,name='qc_h')

# apply H-gate to the 0th qubit
qc_h.h(0)
# qc.save_statevector()
# qc.save_unitary()
qc_h.draw(output='mpl')
```



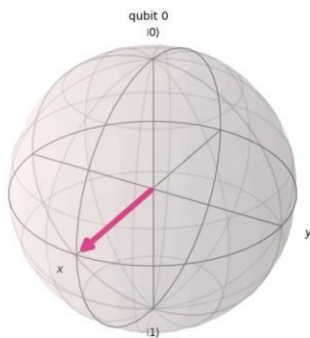
- Measurement (1000 shots):** ~500 results of '0', ~500 results of '1'

```
backend = Aer.get_backend('statevector_simulator')
transpiled_qc_h = transpile(qc_h, backend)
result = backend.run(transpiled_qc_h).result().get_statevector(qc_h, decimals=3)

print("Result after implementing H-gate:\n",result)

plot_bloch_multivector(result)
```

Result after implementing H-gate:
Statevector([0.707+0.j, 0.707+0.j],
dims=[2,])



- **Unitary Gate:**

```
backend = Aer.get_backend('unitary_simulator')
result = backend.run(transpiled_qc_h).result().get_unitary(qc_h, decimals=3)
print("Unitary matrix after implementing H-gate:\n",result)
```

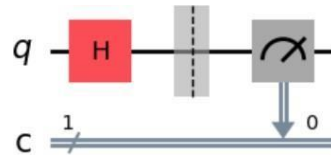
Unitary matrix after implementing H-gate:
 Operator([[0.707+0.j, 0.707-0.j],
 [0.707+0.j, -0.707+0.j]],
 input_dims=(2,), output_dims=(2,))

- **Measurement:**

```
# Measuring the qubit |0> after applying Y-gate

qc_h = QuantumCircuit(1,1,name='qc_y')
qc_h.h(0)
qc_h.barrier()
qc_h.measure(0,0) # measure the qubit

qc_h.draw(output='mpl')
```



- **QASM Simulator:**

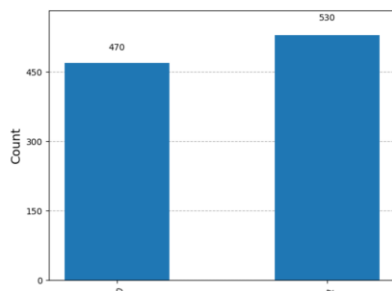
```
backend = Aer.get_backend('qasm_simulator')
job = transpile(qc_h, backend)

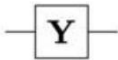
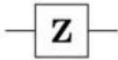
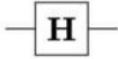
result = backend.run(job, shots=1000).result()
counts = result.get_counts(qc_h)

print("Counts after applying H-gate and measuring the qubit:",counts)

plot_histogram(counts)
```

Counts after applying H-gate and measuring the qubit: ('1': 530, '0': 470)



Operator	Gate(s)	Matrix
Pauli-X (X)	 	$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$
Pauli-Y (Y)		$\begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$
Pauli-Z (Z)		$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$
Hadamard (H)		$\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$

5. Truth Table

Gate	Action on $ 0\rangle$	Action on $ 1\rangle$	Remarks
X	$ 1\rangle$	$ 0\rangle$	Bit – flip (like classical not)
Y	$i 1\rangle$	$-i 0\rangle$	Bit + phase flip
Z	$ 0\rangle$	$- 1\rangle$	Phase flip
H	$(0\rangle + 1\rangle) / \sqrt{2}$	$(0\rangle - 1\rangle) / \sqrt{2}$	Creates superposition

Initial State	Gate Applied	Final State	Measurement Outcome (Probabilities)
$ 0\rangle$	X	$ 1\rangle$	1 with 100% probability
$ 1\rangle$	X	$ 0\rangle$	0 with 100% probability
$ 0\rangle$	Y	$i 1\rangle$	1 with 100% probability
$ 1\rangle$	Y	$-i 0\rangle$	0 with 100% probability
$ 0\rangle$	Z	$ 0\rangle$	0 with 100% probability
$ 1\rangle$	Z	$- 1\rangle$	1 with 100% probability
$ 0\rangle$	H	$(0\rangle + 1\rangle) / \sqrt{2} = +\rangle$	0 $\approx$ 50%, 1 $\approx$ 50%
$ 1\rangle$	H	$(0\rangle - 1\rangle) / \sqrt{2} = -\rangle$	0 $\approx$ 50%, 1 $\approx$ 50%

6. Result

The simulations confirmed the theoretical behaviours of all single-qubit gates:

- **X-gate** flipped the qubit.
- **Y-gate** added a phase but resulted in the same measurement as X.
- **Z-gate** added a phase without affecting measurement.
- **H-gate** placed qubits into superposition, yielding probabilistic measurements.

Conclusion : This lab successfully introduced fundamental quantum computing operations using Qiskit. The theoretical understanding of single-qubit gates was validated through accurate simulations, laying a strong foundation for more complex quantum operations in future labs.